МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
**“КАЗАНСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ им. А.Н.
ТУПОЛЕВА-КАИ”**

Институт **компьютерных технологий и защиты информации**
Кафедра **Систем Информационной безопасности**

Отчет по лабораторной работе №3
По теме
“Функциональная декомпозиция, глобальная сумма и критические секции”
По дисциплине
“Аппаратные средства телекоммуникационных систем”

Выполнил студент
Группы 4404
Сайдаков Л.И.

Казань, 2026

Цель работы. Изучение работы потоков с общей (глобальной) переменной и механизмов взаимного исключения: критических секций и гонки данных. Задача решается методом функциональной декомпозиции: потоки вычисляют разные части формулы и добавляют результаты в общую сумму. В ходе работы выполнена задача 6 методических указаний «Параллельное программирование на OpenMP». Работа выполняется на языке C# (платформа .NET), поэтому конструкции OpenMP моделируются стандартными средствами языка: объекты System.Threading.Thread соответствуют команде потоков, вызов Thread.Join — неявному барьеру в конце параллельной области, оператор lock (класс Monitor) — критической секции (директиве critical), локальные переменные потоков с последующим объединением — операции reduction.

Задание:

Используя функциональную декомпозицию, перепишите задачу 5 для вычисления значения по формуле (Σx + Σy + Σz) / 3N, где N — количество точек. Используйте глобальную сумму и критические секции.

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, целиком решающие задачу (в частности, Parallel.For и PLINQ); каждый запуск, о котором говорится в задаче (разные режимы программы), подтверждается скриншотом с видимой командой запуска и полным выводом.

Ход работы. Координаты точек считываются из текстового файла (по одной точке в строке, три числа x, y, z через пробел) в три массива X, Y и Z, как и в задаче 5. Режим gen создаёт файл со случайными точками для больших входных данных.

Вычисление построено на функциональной декомпозиции: создаются три потока, каждый из которых отвечает за одну координату — первый суммирует x, второй y, третий z. Все три потока добавляют свои слагаемые в одну общую переменную — глобальную сумму. Итоговое значение вычисляется после завершения потоков (Join) как отношение глобальной

суммы к 3N, где N — количество точек. Результат сравнивается с последовательным вычислением по той же формуле с относительной погрешностью 10^{-9} (порядок сложения чисел с плавающей точкой в потоках отличается, поэтому последние разряды могут различаться).

Так как три потока одновременно изменяют одну переменную, обращение к ней необходимо защитить. Операция total += v состоит из чтения, сложения и записи и не является атомарной: два потока могут прочитать одно и то же старое значение, и результат одного из них будет потерян (гонка данных). Для этого используется критическая секция — оператор lock, в который в каждый момент времени может войти только один поток. В программе реализованы три режима, выбираемые аргументом командной строки:

— critical: критическая секция выполняется на каждое слагаемое (прямой аналог директивы critical внутри цикла); результат верный, но потоки постоянно ждут друг друга на блокировке, поэтому режим самый медленный;

— local: каждый поток накапливает сумму своей координаты в локальной переменной и входит в критическую секцию один раз, чтобы добавить её к глобальной сумме; число входов в критическую секцию равно трём, поэтому режим существенно быстрее critical;

— race: то же вычисление без синхронизации; режим нужен только для демонстрации гонки данных, результат не гарантирован и на больших файлах, как правило, не совпадает с эталоном (на малых файлах гонка может не проявиться).

Команды запуска:

```
cd lab4
dotnet run -c Release -- gen points.txt 10000000
dotnet run -c Release -- sample_points.txt
dotnet run -c Release -- points.txt critical
dotnet run -c Release -- points.txt local
dotnet run -c Release -- points.txt race
dotnet run -c Release -- points.txt all
```

На рисунке 1 показан запуск на небольшом тестовом файле sample_points.txt (вершины единичного куба). Сумма всех координат равна 12, N = 8, поэтому результат (12 / 24) равен 0.5; все режимы выдали это значение, так как на малом числе точек гонка данных не проявилась.

```
PS U:\111\parallel_programming\Lab4> dotnet run -c Release -- gen points.txt 10000000
Записано точек: 10000000, файл: points.txt
PS U:\111\parallel_programming\Lab4> dotnet run -c Release -- sample_points.txt
Файл: sample_points.txt, точек: 8
Последовательно: сумма = 12.000000, результат = 0.500000000
Режим critical: сумма = 12.000000, результат = 0.500000000, время = 2.885 мс, совпадает: True
Режим local   : сумма = 12.000000, результат = 0.500000000, время = 0.985 мс, совпадает: True
Режим race    : сумма = 12.000000, результат = 0.500000000, время = 0.783 мс, совпадает: True
Примечание: в режиме race результат не гарантирован; для малых N гонка может не проявиться.
OK: True
PS U:\111\parallel_programming\Lab4>
```

Рис. 1 – запуск на тестовом файле sample_points.txt, все режимы

На рисунке 2 показан запуск режима critical на файле из 10 000 000 точек. Видны команда запуска, глобальная сумма, результат, время выполнения и совпадение с эталоном.

```
PS U:\111\parallel_programming\Lab4> dotnet run -c Release -- points.txt critical
Файл: points.txt, точек: 10000000
Последовательно: сумма = -145313.982073, результат = -0.004843799
Режим critical: сумма = -145313.982073, результат = -0.004843799, время = 1715.059 мс, совпадает: True
OK: True
PS U:\111\parallel_programming\Lab4>
```

Рис. 2 – режим critical (критическая секция на каждое слагаемое)

На рисунке 3 показан запуск режима local на том же файле. Результат совпадает с эталоном, а время выполнения меньше, чем в режиме critical, так как критическая секция выполняется всего три раза.

```
PS U:\111\parallel_programming\Lab4> dotnet run -c Release -- points.txt local
Файл: points.txt, точек: 10000000
Последовательно: сумма = -145313.982073, результат = -0.004843799
Режим local   : сумма = -145313.982073, результат = -0.004843799, время = 47.056 мс, совпадает: True
OK: True
PS U:\111\parallel_programming\Lab4>
```

Рис. 3 – режим local (локальные суммы и одна критическая секция на поток)

На рисунке 4 показан запуск режима race без синхронизации. Из-за потерянных обновлений глобальной суммы результат не совпадает с эталоном, что демонстрирует необходимость критической секции.

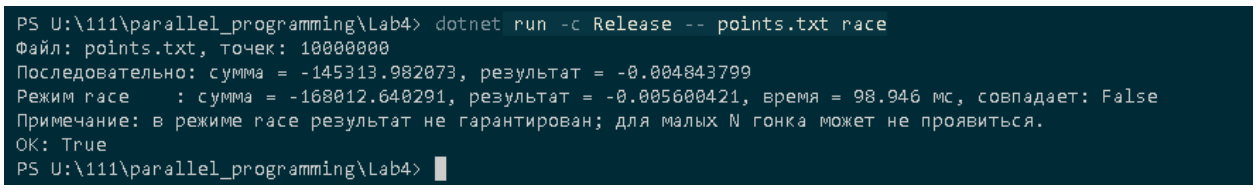


Рис. 4 – режим race (без синхронизации, демонстрация гонки данных)

На рисунке 5 показан запуск всех режимов в одном процессе. Сводные результаты приведены в таблице 1.

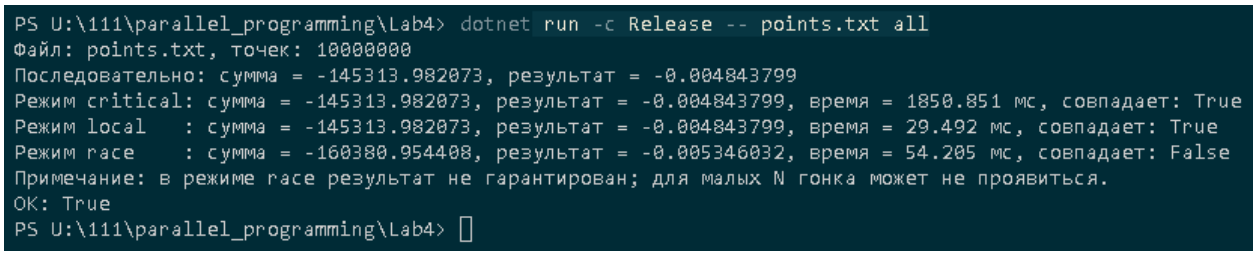


Рис. 5 – запуск всех режимов (all) на файле из 10 000 000 точек

Таблица 1 — Сравнение режимов вычисления глобальной суммы

Режим	**Результат**	**Время, мс**	**Совпадает с эталоном**
Последовательно	-0.004843799	-	-
critical	-0.00484379	1850.851	True
local	-0.004843799	29.492	True
race	-0.005346032	54.205	False

Ответы на вопросы

Зачем нужна критическая секция? Несколько потоков одновременно изменяют общую переменную; операция сложения с присваиванием не атомарна, поэтому без взаимного исключения часть обновлений теряется и результат становится недетерминированным. Критическая секция допускает в защищённый участок только один поток одновременно.

Чем отличаются режимы critical и local? В режиме critical блокировка захватывается на каждое слагаемое (порядка 3N раз), и потоки в основном ждут друг друга. В режиме local блокировка захватывается один раз на поток, а основная работа выполняется в локальных переменных без синхронизации, поэтому накладные расходы минимальны.

Почему результат параллельных режимов может отличаться от последовательного в последних разрядах? Сложение чисел с плавающей точкой неассоциативно: при другом порядке слагаемых округления накапливаются иначе, поэтому для сравнения используется относительная погрешность, а не строгое равенство.

Выводы

1. Методом функциональной декомпозиции реализовано вычисление значения (Σx + Σy + Σz) / 3N с использованием глобальной суммы и критических секций (оператор lock).

2. Режимы critical и local дают верный результат, совпадающий с последовательным вычислением. Режим race без синхронизации демонстрирует гонку данных и потерю обновлений.

3. Количество входов в критическую секцию сильно влияет на производительность: накопление локальной суммы и одно обновление глобальной переменной на поток (режим local) существенно эффективнее, чем захват блокировки на каждое слагаемое (режим critical).

4. Функциональная декомпозиция в данной задаче использует три потока (по числу координат), поэтому ускорение ограничено тремя.